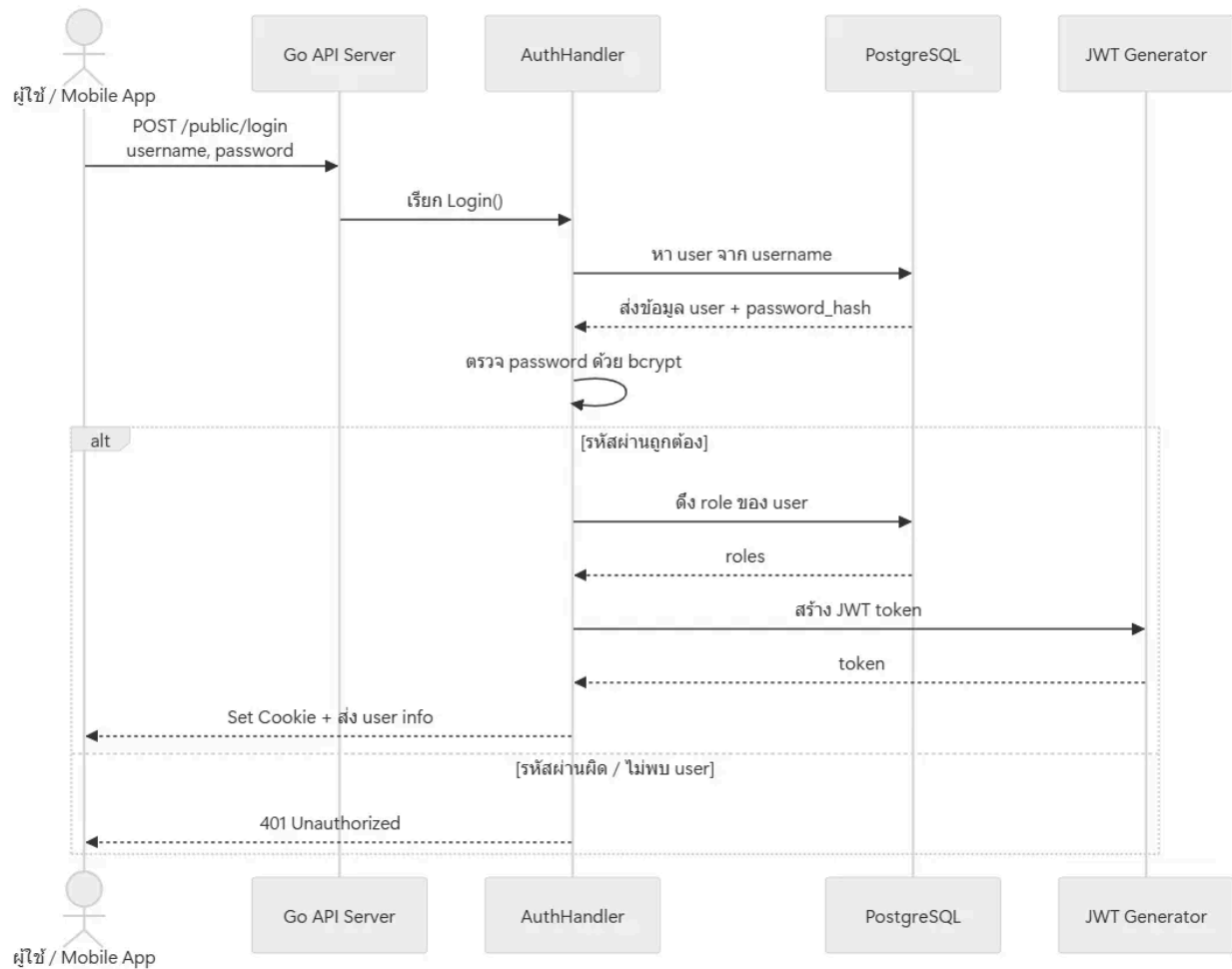
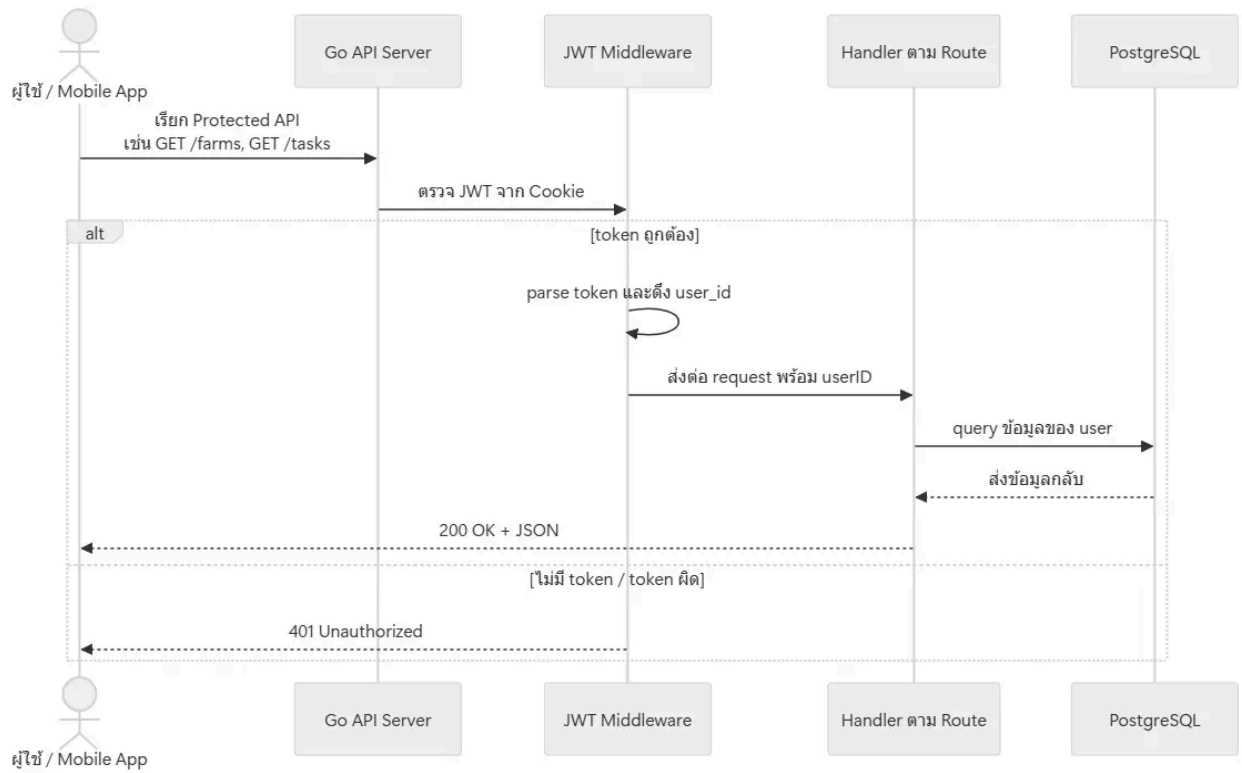


sequence diagram go

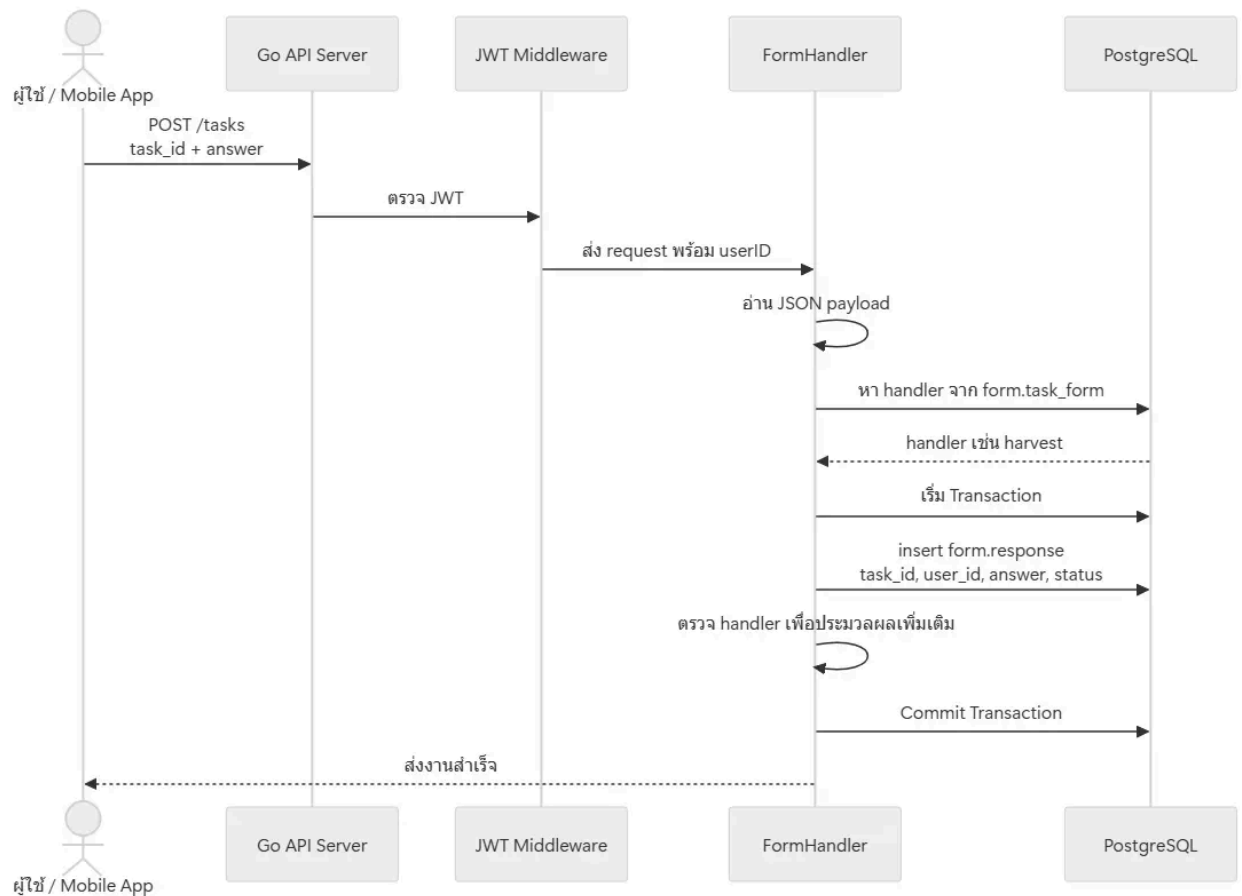
1. Login และได้ Token



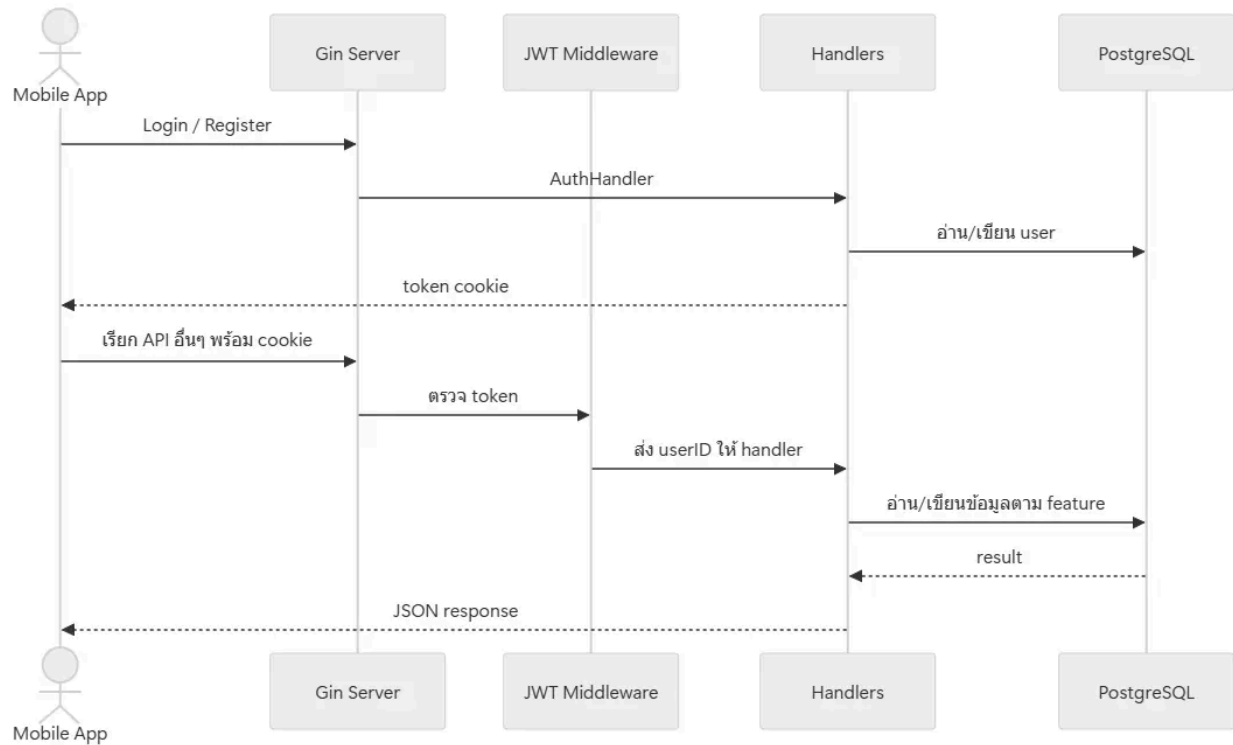
2. เรียก API ที่ต้อง Login ก่อน



3. Flow ผลงาน / Dynamic Form



ภาพรวมสั้นๆ



สรุปง่ายๆ คือ **Mobile App** คุยกับ **Gin Server** → ถ้าเป็น **route protected** จะผ่าน **JWT Middleware** ก่อน → **Handler** ใช้ **userID** ไป **query** หรือบันทึกข้อมูลใน **PostgreSQL** → ส่ง **JSON** กลับไปให้แอป.

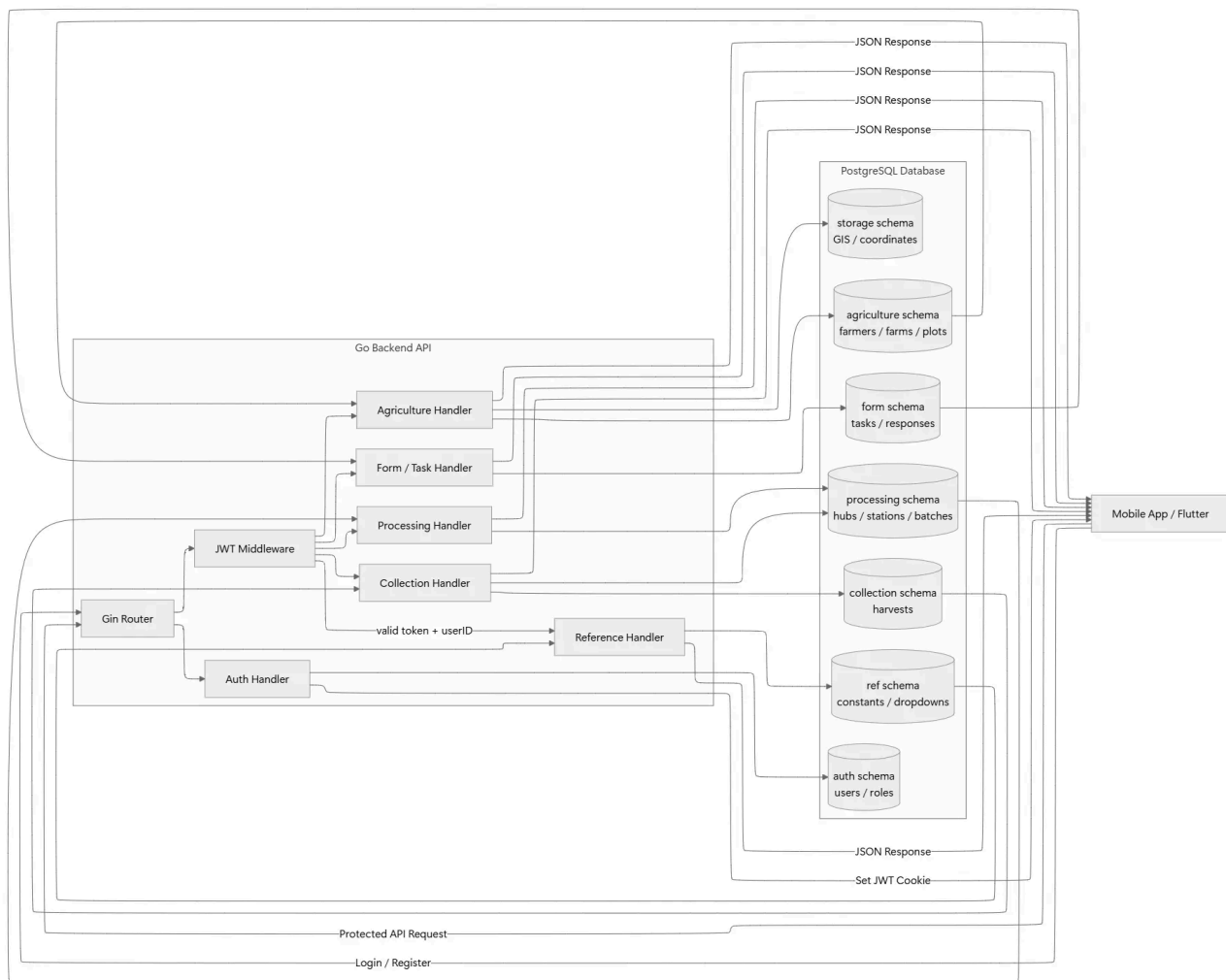
flutter App ส่ง request เข้า **Go Backend** ผ่าน **Gin Router**

ถ้าเป็น API ที่ต้อง login จะผ่าน **JWT Middleware** ก่อน

จากนั้นค่อยส่งต่อไปยัง handler แต่ละกลุ่ม เช่น farm, hub, processing, task

แล้ว handler จะอ่าน/เขียนข้อมูลใน **PostgreSQL** ตาม schema ที่เกี่ยวข้อง ก่อนส่ง **JSON** กลับไป

ให้แอป.



ไฟล์ทำอะไร / มีฟังก์ชันอะไร / API เส้นไหนทำอะไร

ภาพรวมโครงสร้าง

cmd/ main.go	จุดเริ่มต้น server + กำหนด routes ทั้งหมด
internal/ database/ postgres.go	เชื่อมต่อ PostgreSQL ด้วย GORM
middleware/ auth_middleware.go	ตรวจ JWT ก่อนเข้า protected API
handlers/ auth_handler.go agriculture_handler.go collection_handler.go processing_handler.go form_handler.go	สมัครสมาชิก, login, ข้อมูลผู้ใช้ farmer, farm, plot hub collector, hub, harvest processor, processing station, batch task / dynamic form / submit form

ref_handler.go	dropdown / constants / reference data
models/	
auth.go	user account + login/register request
farmer.go	โครงสร้างข้อมูล farmer
farm.go	โครงสร้างข้อมูล farm
plot.go	โครงสร้างข้อมูล plot
hub.go	โครงสร้างข้อมูล hub
harvest.go	โครงสร้างข้อมูล harvest
batch.go	โครงสร้างข้อมูล batch
processing_station.go	โครงสร้างข้อมูล processing station
ref.go	response struct สำหรับ constants/dropdowns

cmd/main.go

หน้าที่หลัก: start server, connect database, สร้าง handler, กำหนด API routes

ฟังก์ชัน:

```
main()
```

สิ่งที่ทำ:

1. โหลด .env
2. เชื่อมต่อ database ผ่าน database.InitDB()
3. สร้าง handler ต่างๆ
4. สร้าง Gin router
5. แบ่ง routes เป็น public และ protected
6. เปิด server ที่ port 8080

API ที่ประกาศในไฟล์นี้ทั้งหมด:

```
Public API
POST /public/login
POST /public/register
GET /public/test

Protected API ต้องผ่าน JWT
GET /auth/me
GET /constants/:key

POST /farmers
POST /farms
POST /plots
GET /farms
GET /plots
```

```
POST /hub_collectors
POST /hubs
GET /hubs
GET /harvests

POST /processors
POST /processing_stations
GET /processing_stations
GET /batches

GET /tasks
POST /tasks
GET /tasks/:taskId
PUT /tasks
```

internal/database/postgres.go

หน้าที่หลัก: เชื่อมต่อ PostgreSQL

ฟังก์ชัน:

```
InitDB()
```

ทำอะไร:

1. โหลดค่า DB จาก .env
2. ประกอบ connection string
3. เปิด connection ด้วย GORM
4. ตั้งค่า connection pool
5. return *gorm.DB ให้ handler ใช้งาน

internal/middleware/auth_middleware.go

หน้าที่หลัก: ตรวจสอบว่า request มี JWT cookie ถูกต้องไหม

ฟังก์ชัน:

```
JwtAuthMiddleware()
```

ทำอะไร:

1. อ่าน JWT จาก cookie ตามชื่อ JWT_NAME
2. ตรวจสอบ token ด้วย JWT_KEY
3. ดึง user_id จาก token
4. เก็บ userID เข้า gin.Context
5. ถ้า token ไม่ถูกต้อง ตอบ 401

ใช้กับ API ทุกเส้นที่อยู่ใน protected group

internal/handlers/auth_handler.go

หน้าที่หลัก: ระบบ authentication

Struct:

```
AuthHandler
```

ฟังก์ชัน:

```
GenerateToken(userID, username)  
Login(c)  
Register(c)  
GetMe(c)
```

รายละเอียด:

```
GenerateToken()  
สร้าง JWT token โดยฝัง user_id, username, issued_at, expired_at  
  
Login()  
ตรวจ username/password, ดึง roles, สร้าง JWT, set cookie  
  
Register()  
สมัคร user ใหม่, hash password, insert ลง auth.user_account  
  
GetMe()  
ดึงข้อมูล profile ของ user ปัจจุบัน พร้อม role
```

API:

```
POST /public/login  
ใช้ Login()  
ทำหน้าที่ login และ set JWT cookie  
  
POST /public/register  
ใช้ Register()  
ทำหน้าที่สมัครสมาชิก  
  
GET /auth/me  
ใช้ GetMe()  
ทำหน้าที่ดึงข้อมูล user ที่ login อยู่
```

internal/handlers/agriculture_handler.go

หน้าที่หลัก: จัดการข้อมูลฝั่งเกษตรกร เช่น farmer, farm, plot

Struct:

```
AgricultureHandler
RegisterFarmerProfileRequest
RegisterFarmRequest
RegisterPlotRequest
```

ฟังก์ชัน:

```
_parseDate(dateStr)
RegisterFarmerProfile(c)
RegisterFarm(c)
RegisterPlot(c)
GetMyFarms(c)
GetMyPlots(c)
```

รายละเอียด:

```
_parseDate()
แปลง string เป็น time.Time

RegisterFarmerProfile()
ลงทะเบียน profile เกษตรกร และเพิ่ม role farmer

RegisterFarm()
สร้าง farm, ผูก farm กับ farmer, บันทึก GIS ถ้ามี

RegisterPlot()
สร้าง plot, บันทึก GIS polygon ถ้ามี

GetMyFarms()
ดึง farm ของ user ปัจจุบัน พร้อม plots

GetMyPlots()
ดึง plots ของ user ปัจจุบัน
```

API:

```
POST /farmers
ลงทะเบียนข้อมูลเกษตรกร

POST /farms
สร้างฟาร์มของเกษตรกร

POST /plots
```

สร้างแปลงปลูกในฟาร์ม

GET /farms

ดึงฟาร์มของ user ที่ login อยู่

GET /plots

ดึงแปลงปลูกของ user ที่ login อยู่

internal/handlers/collection_handler.go

หน้าที่หลัก: จัดการ hub collector, hub, harvest

Struct:

```
CollectionHandler
RegisterHubCollectorRequest
RegisterHubRequest
```

ฟังก์ชัน:

```
RegisterHubCollector(c)
RegisterHub(c)
GetMyHub(c)
GetMyHarvests(c)
```

รายละเอียด:

```
RegisterHubCollector()
ลงทะเบียน user เป็น hub collector และเพิ่ม role hub_collector

RegisterHub()
สร้าง hub, ผูก hub กับ collector, บันทึก GIS ถ้ามี

GetMyHub()
ดึง hub ของ collector พร้อม harvests ใน hub นั้น

GetMyHarvests()
ดึง harvests ที่อยู่ภายใต้ hub ของ collector
```

API:

```
POST /hub_collectors
ลงทะเบียน user เป็นผู้รวบรวมผลผลิต

POST /hubs
สร้าง hub / จัดรวบรวมผลผลิต
```

GET /hubs

ดึง hub ของ user พร้อมรายการ harvest

GET /harvests

ดึงรายการ harvest ที่ user มีสิทธิ์ดู

internal/handlers/processing_handler.go

หน้าที่หลัก: จัดการ processor, processing station, batch

Struct:

```
ProcessingHandler  
RegisterProcessorRequest  
RegisterStationRequest
```

ฟังก์ชัน:

```
RegisterProcessor(c)  
RegisterStation(c)  
GetMyProcessingStation(c)  
GetMyBatches(c)
```

รายละเอียด:

```
RegisterProcessor()  
ลงทะเบียน user เป็น processor และเพิ่ม role processor  
  
RegisterStation()  
สร้าง processing station และผูกกับ processor  
  
GetMyProcessingStation()  
ดึงสถานะแปรรูปของ processor พร้อม batches  
  
GetMyBatches()  
ดึง batch ที่อยู่ในสถานะของ processor
```

API:

```
POST /processors  
ลงทะเบียน user เป็นผู้แปรรูป  
  
POST /processing_stations  
สร้างสถานะแปรรูป  
  
GET /processing_stations  
ดึงสถานะแปรรูปของ user
```

GET /batches

ดึง batch ของสถานที่ user ดูแล

internal/handlers/form_handler.go

หน้าที่หลัก: จัดการ task และ dynamic form submission

Struct:

FormHandler

SubmitFormRequest

ฟังก์ชัน:

GetTasks(c)

SubmitTask(c)

GetTaskResponse(c)

UpdateTaskResponse(c)

รายละเอียด:

GetTasks()

ดึงรายการ tasks พร้อมสถานะ เช่น COMPLETED, OVERDUE, NOT_STARTED

SubmitTask()

ส่งคำตอบของ task, insert ลง form.response

GetTaskResponse()

ดึงคำตอบเดิมของ task ตาม taskId

UpdateTaskResponse()

แก้ไขคำตอบของ task ที่เคยส่งแล้ว

API:

GET /tasks

ดึงรายการงานทั้งหมดของ user

รองรับ query: ?date=YYYY-MM-DD

POST /tasks

ส่งคำตอบของ task

body มี task_id และ answer

GET /tasks/:taskId

ดึงคำตอบที่เคยส่งของ task นั้น

```
PUT /tasks
แก้ไขคำตอบ task เดิม
body มี task_id และ answer
```

internal/handlers/ref_handler.go

หน้าที่หลัก: ดึงข้อมูล dropdown / constants / reference data ให้ frontend

Struct:

```
RefHandler
```

ฟังก์ชัน:

```
GetConstants(c)
```

API:

```
GET /constants/:key
```

ตัวอย่าง key ที่ใช้ได้:

```
air_exposure_type
breed
chem_bio
cocoa_brean_grade
drying_facility
farm_activity_type
fertilizer_stage
fertilizer
grade
hole_filler
land_type
location_type
pest_disease
processing_activity_type
processing_defect
soil_type
tank_material
water_source
watering_system
weather_condition
province
district
subdistrict
farm
plot
```

hub
processing_station
batch
harvest
location

ตัวอย่างการเรียก:

GET /constants/province

ดึงจังหวัดทั้งหมด

GET /constants/district?province_id=xxx

ดึงอำเภอตามจังหวัด

GET /constants/subdistrict?district_id=xxx

ดึงตำบลตามอำเภอ

GET /constants/farm

ดึง farm ของ user เพื่อใช้เป็น dropdown

GET /constants/plot?farm_id=xxx

ดึง plot ใน farm นั้น

GET /constants/location?zip_code=10110

ดึงข้อมูล location จากรหัสไปรษณีย์

models แต่ละไฟล์

auth.go

เก็บ UserAccount, LoginRequest, RegisterRequest

ใช้กับ auth.user_account

farmer.go

เก็บข้อมูลเกษตรกร เช่น ชื่อ เบอร์ ที่อยู่ role/profile

farm.go

เก็บข้อมูลฟาร์ม เช่น farm_id, farm_name, address, contact

มี relation ไป plots

plot.go

เก็บข้อมูลแปลงปลูก เช่น plot_id, farm_id, area, land type, soil type

hub.go

เก็บข้อมูล hub / จดรวบรวม

มี relation ไป harvests

harvest.go

เก็บข้อมูล harvest / ผลผลิตที่เก็บเกี่ยว

processing_station.go

เก็บข้อมูลสถานีแปรรูป

มี relation ไป batches

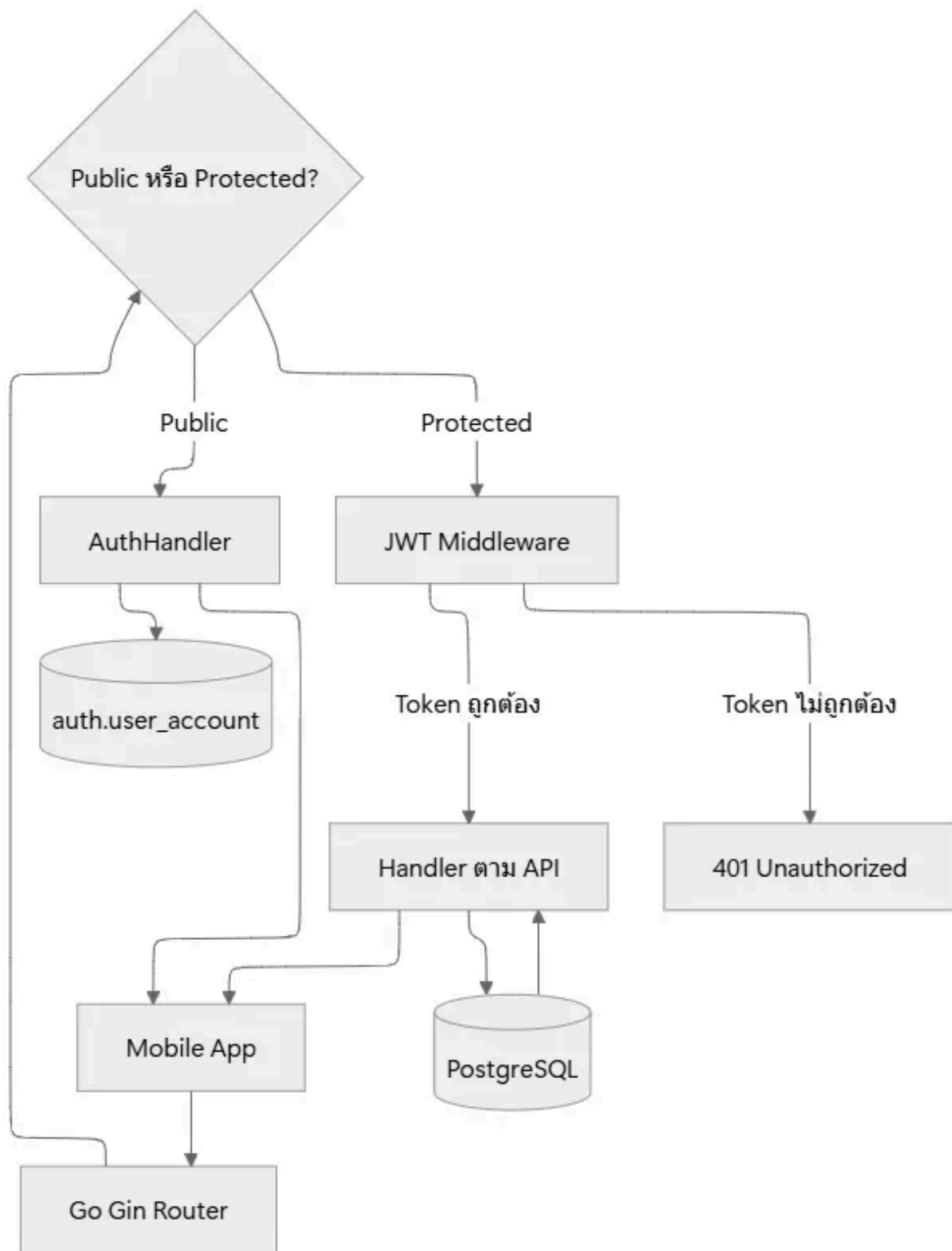
batch.go

เก็บข้อมูล batch ในสถานีแปรรูป เช่น origin, quantity, notes

ref.go

เก็บ struct response สำหรับ constants/dropdowns เช่น province, district, soil type, farm dropdown

สรุป Flow หลักของระบบ



สรุป API แบบสั้น

Auth	
POST /public/register	สมัครสมาชิก
POST /public/login	เข้าสู่ระบบ
GET /auth/me	ดูข้อมูลตัวเอง

Reference

GET /constants/:key ดึง dropdown/constants

Agriculture

POST /farmers ลงทะเบียนเกษตรกร

POST /farms สร้างฟาร์ม

POST /plots สร้างแปลงปลูก

GET /farms ดูฟาร์มของตัวเอง

GET /plots ดูแปลงของตัวเอง

Collection

POST /hub_collectors ลงทะเบียนผู้รวบรวม

POST /hubs สร้าง hub

GET /hubs ดู hub ของตัวเอง

GET /harvests ดู harvest ของ hub ตัวเอง

Processing

POST /processors ลงทะเบียนผู้แปรรูป

POST /processing_stations สร้างสถานีแปรรูป

GET /processing_stations ดูสถานีแปรรูปของตัวเอง

GET /batches ดู batch ของตัวเอง

Tasks / Forms

GET /tasks ดูรายการ task

POST /tasks ส่งคำตอบ task

GET /tasks/:taskId ดูคำตอบ task เดิม

PUT /tasks แก้ไขคำตอบ task